Paradigmas de Programação — Aula 09: Modelo de atores com Elixir

GenServer e Supervisor

OTP e GenServer

- **OTP**: bibliotecas e padrões de processo que acompanham a BEAM
- **Comportamento** (*behaviour*): módulo genérico que faz a parte comum e chama *callbacks* do programador

Callback	**Trata**	**Devolve**
`init(arg)`	início do processo	`{:ok, estado}`
`handle_call(pedido, de, estado)`	pedido **síncrono** (`GenServer.call`)	`{:reply, resposta, novo_estado}`
`handle_cast(pedido, estado)`	pedido **assíncrono** (`GenServer.cast`)	`{:noreply, novo_estado}`
`handle_info(msg, estado)`	mensagem fora de `call` e `cast`	`{:noreply, novo_estado}`

- `call` bloqueia até a resposta (prazo padrão 5000 ms); `cast` retorna `:ok` de imediato

A memória como GenServer (`05-genserver.exs`)

```elixir
defmodule Memoria do
  use GenServer

  # API do cliente: executa no processo de quem chama
  def start_link(inicial) do
    GenServer.start_link(__MODULE__, inicial, name: __MODULE__)
  end
  def salvar(valor), do: GenServer.cast(__MODULE__, {:salvar, valor})
  def ler, do: GenServer.call(__MODULE__, :ler)

  # Callbacks: executam no processo da memória
  @impl true
  def init(inicial), do: {:ok, inicial}
  @impl true
  def handle_cast({:salvar, novo_valor}, _valor), do: {:noreply, novo_valor}
  @impl true
  def handle_call(:ler, _do_processo, valor), do: {:reply, valor, valor}
end
```

Saída: `antes: 0`, `depois: 42`

Do laço `receive` ao GenServer

`03` (primitivas)	`05` (GenServer)
`spawn(fn -> Memoria.loop(0) end)`	`Memoria.start_link(0)` → `init(0)`
argumento `valor` de `loop/1`	estado, último argumento do *callback*
`{:salvar, novo_valor, do_processo} -> ...; loop(novo_valor)`	`handle_cast` → `{:noreply, novo_valor}`
`send(memoria, {:ler, self()})` + `receive`	`GenServer.call(__MODULE__, :ler)`
`{:ler, do_processo} -> send(do_processo, {:valor, valor}); loop(valor)`	`handle_call` → `{:reply, valor, valor}`
chamada recursiva `loop(novo_valor)`	tupla de retorno; o OTP faz a recursão
PID em variável	nome registrado `Memoria`
cláusula `mensagem_desconhecida`	`handle_info/2`

- Sem confirmação do `:salvar`: os `cast` e o `call` saem do **mesmo** processo para o **mesmo** destino

Let it crash e registro por nome

- **Let it crash** (Armstrong, 2003): o processo do trabalho não trata toda falha possível; diante do que não sabe resolver, **termina**
- Outro processo, que observa, decide o que fazer — em geral, reiniciar em estado inicial conhecido
- Código do trabalho sem `try` / `rescue` para falhas que não sabe tratar; recuperação concentrada em um só lugar
- Viável porque processos não compartilham estado: a queda de um não corrompe outro
- `name: __MODULE__`: o cliente endereça o **nome**; o processo reiniciado tem **PID novo**, e o nome continua valendo

Supervisor: queda e reinício (`06-supervisor.exs`)

```
filhos = [{Memoria, 0}]
{:ok, _supervisor} = Supervisor.start_link(filhos, strategy: :one_for_one)
# handle_cast({:salvar, novo_valor}, _) when is_number(novo_valor)
Memoria.salvar(42)
Memoria.salvar("quarenta e dois")      # nenhuma cláusula casa: queda
```

```
valor guardado: 42
memoria caiu: :function_clause
PID antes: #PID<0.103.0>, PID depois: #PID<0.104.0>
valor guardado: 0
```

- **O estado morre com o processo**: o reinício executa `init(0)` de novo
- O reinício devolve um processo correto, não o estado anterior; estado que precisa sobreviver fica fora do processo que cai

A árvore de supervisão

```
flowchart TB
    S["Supervisor<br/>strategy: :one_for_one"]
    S -->|"inicia: Memoria.start_link(0)"| A["Memoria, PID 0.103.0<br/>estado: 42"]
    A -->|"salvar com texto<br/>:function_clause"| X(("queda"))
    X -->|"sinal de saída"| S
    S -->|"reinicia: Memoria.start_link(0)"| B["Memoria, PID 0.104.0<br/>estado: 0"]
```

- O Supervisor recebe o sinal de saída do filho e o inicia de novo com o mesmo argumento
- Mesmo nome, PID novo, estado inicial: o 42 se perdeu com o processo anterior

Estratégias, tipos de reinício e limite

Estratégia	Quando um filho termina
`:one_for_one`	só ele é reiniciado
`:one_for_all`	todos são terminados e reiniciados
`:rest_for_one`	ele e os iniciados depois dele

Tipo	Reinicia quando
`:permanent` (padrão)	sempre
`:transient`	só em término anormal (não `:normal`, `:shutdown`, `{:shutdown, _}`)
`:temporary`	nunca

- Reinício resolve falha **transitória**; defeito **determinístico** cai de novo
- `max_restarts`: padrão de 3 reinícios em 5 segundos; excedido, o Supervisor termina os filhos e a si mesmo, e a falha sobe na árvore